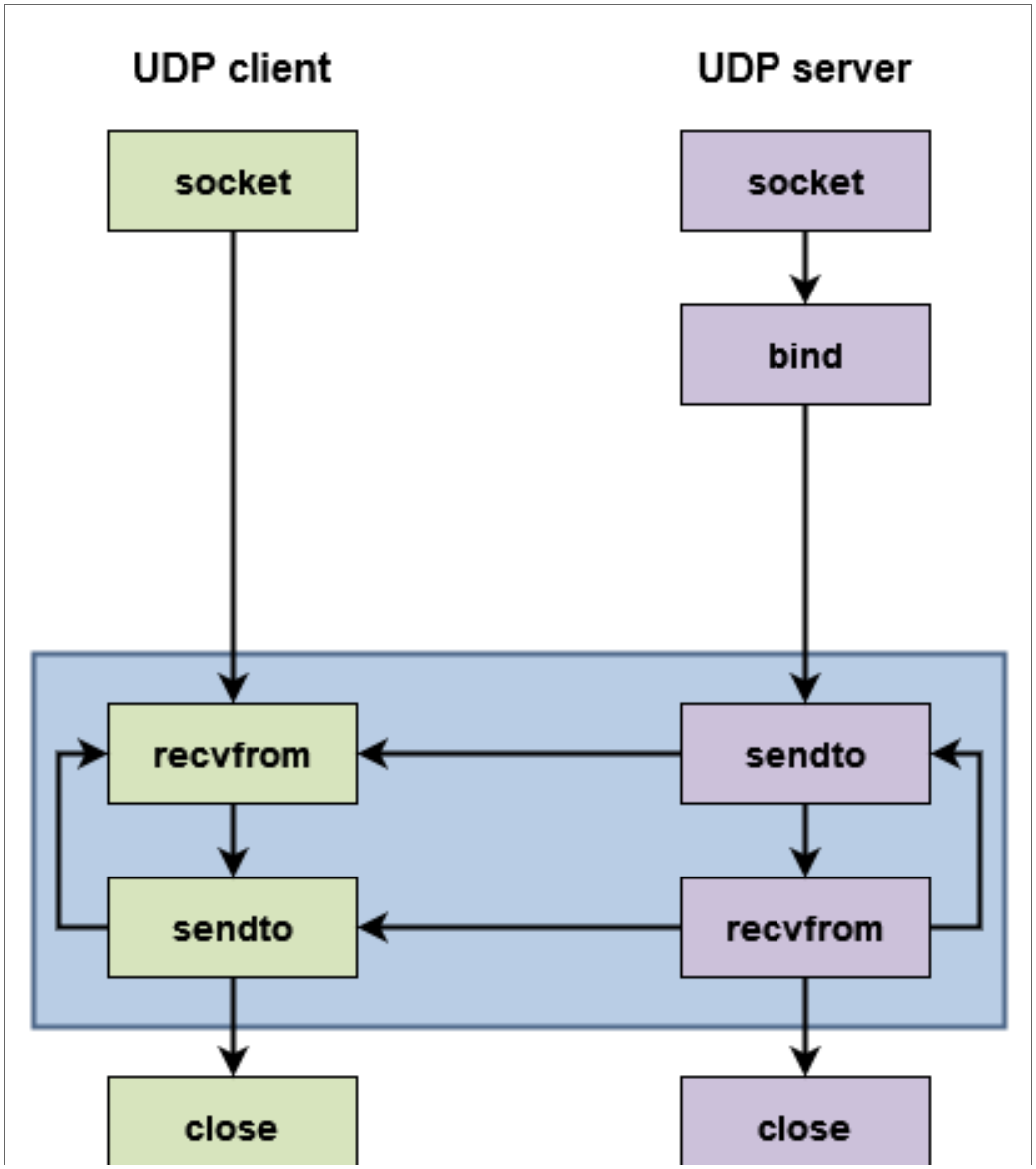
Telecommunications Network

Practice 5

UDP


UDP client
UDP server
socket
socket
bind
recvfrom
sendto
sendto
recvfrom
close
close



UDP

- socket

```
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

- recvfrom()

```
data, address = sock.recvfrom(4096)
```

- sendto()

```
sent = sock.sendto(data, address)
```

Exercise I.

- Write a server-client application that uses UDP.
- The client should send a b”Hello Server” bytestring to the server and in return the server should respond with b”Hello Client”.
- Is our server capable of handling multiple clients? Why or why not?

Netmask (Subnets)

Creating subnets

Source: **https://www.seomining.com/web-technologies/module4/purpose-subnetting-subnetMasks.php** and **https://bytesofgigabytes.com/networking/what-is-subnetting-in-networking/**

Source: **https://en.wikipedia.org/wiki/Subnetwork**

Netmask

- Description of addresses in a subnet.
- Network + broadcast address: 2 reserved addresses.

Netmask RFC
CIDR RFC

Subnet mask

- A subnet is a logical subdivision of an IP network. An IP address therefore consists of two parts: the network number and the host identifier.
- The two parts are separated with the 32-bit subnet mask: a bitwise AND of the mask and the IP address gives the network identifier, and a bitwise AND with the complement of the mask gives the host identifier.
- This lets us decide whether the destination host is on the local subnet or not.
- If it is not, the packet is forwarded to the default router (default gateway).

Subnet mask

- CIDR notation: a compact representation of an IP address and its network number
- → IP address + ‘/’ + decimal number.
- E.g. in 135.46.57.14/24, 135.46.57.14 is the IP address,
- 255.255.255.0 is the netmask (24 one-bits counted from the front),
- so 135.46.57.0 is the network identifier.

Subnet mask – example

	10000111	00101110	00111001	00001110	**135.46.57.14**
AND	11111111	11111111	11111111	00000000	**255.255.255.0**
	10000111	00101110	00111001	00000000	**135.46.57.0**

135.46.57.14/24 → 135.46.57.0

Exercise II.

- How many addresses are available with the following subnet masks? Give the minimum and the maximum address as well!
- 188.100.22.12/32
- 188.100.22.12/20
- 188.100.22.12/10

Exercise II. – solution

- 188.100.22.12/32 : a single address, 188.100.22.12
- 188.100.22.12/20 : it would be $2^{32-20} = 2^{12} = 4096$ addresses, but two of them have to be subtracted, because they have a special meaning:
  - all 0s: the network address of the subnet (it refers to the subnet itself)
  - all 1s: broadcast on the local network
- So there are 4094; the min. address is 188.100.16.1, the max. address is 188.100.31.254
- 188.100.22.12/10 : $2^{32-10} - 2 = 4194302$, the min. address is 188.64.0.1, the max. address is 188.127.255.254.

Multicast

Class	Range	Description
A	0.0.0.0 - 127.255.255.255	Unicast
B	128.0.0.0 - 191.255.255.255	Unicast
C	192.0.0.0 - 223.255.255.255	Unicast
D	224.0.0.0 - 239.255.255.255	Multicast
E	240.0.0.0 - 255.255.255.255	Reserved

Multicast

Multicast

- `setsockopt()` (sender)

```
ttl = struct.pack("b", 1)
sock.setsockopt(socket.IPPROTO_IP, socket.IP_MULTICAST_TTL, ttl)
```

- Adding a socket to the multicast group (recv)

```
multicast_group = "224.3.29.71"
group = socket.inet_aton(multicast_group)
mreq = struct.pack("4sL", group, socket.INADDR_ANY)
sock.setsockopt(socket.IPPROTO_IP, socket.IP_ADD_MEMBERSHIP, mreq)
```

Exercise III.

- Modify the calculator application so that it uses UDP instead of TCP.

Exercise IV.

- Write a server-client application, where the client send a picture to the server over UDP.
  - The client should send the file in 200 byte chunks.
  - If it gets to the end of the file, the client should send an empty string.
  - The server should acknowledge every chunk it receives with the b”OK” bytestring.